

F2 (Integração MCP) — Relatório da Fase 3

Geração de Código: o app gerado usa as ferramentas MCP de verdade Data: 24/07/2026 · Commit: 28aef3c

1. Objetivo

Fechar a ponta: o **app gerado** pelo LangNet deve, em runtime, **chamar as ferramentas MCP** atribuídas aos seus agentes (Fase 2) — no mesmo mecanismo de tools que já existe (CrewAI + TOOL_REGISTRY).

2. O que foi implementado (no gerador)

- `_fetch_mcp_assignments(project_id)` — lê as tools MCP atribuídas aos agentes (tabela `mcp_agent_tools`) + dados do servidor (URL, transporte, descrição), só de servidores habilitados.
- **Injeção no `agents.yaml`** — os nomes das tools MCP entram no `agents_map` **antes** da injeção, então cada agente ganha suas tools MCP na lista `tools:`.
- `ws-server/mcp_tools.py` (emitido) — um `BaseTool` do CrewAI por tool MCP, cujo `_run` chama a ferramenta no servidor MCP via cliente `mcp cru` (SSE/HTTP); credenciais vêm de `MCP_CRED_<id>` (env). Exporta `MCP_TOOLS`.
- `ws-server` — mescla `MCP_TOOLS` no `TOOL_REGISTRY`, então os agentes resolvem as tools MCP por nome.
- `requirements.txt` — ganha `mcp>=1.0.0`.

Decisão técnica: NÃO uso o `MCPServerAdapter` do `crewai_tools` — sua detecção do pacote `mcp` está quebrada e dispara um **prompt interativo** ("missing 'mcp' package? [y/N]"), inaceitável num servidor. Usei um **wrapper próprio** com o cliente `mcp cru`, que é robusto e testado.

3. Testes de validação (todos executados)

T1 — Unit test do gerador de `mcp_tools.py`

`mcp_tools.py` gerado: 1891 chars | tem MCP_TOOLS? True | tem enriquecer_lead? True | compila OK



T2 — Regeneração completa do código (com atribuições da Fase 2)

sessão: 2630fd53-... | arquivos: 74
[CODE-GEN] 2 tool(s) MCP atribuída(s) a agentes



(74 arquivos = 1 a mais, o `mcp_tools.py`.)

T3 — Verificação dos artefatos gerados

1. `ws-server/mcp_tools.py` presente? True | tools: ['enriquecer_lead', 'buscar_conta']
2. `lead_identifier_agent` tools: ['database_tool', 'embedding_tool', 'enriquecer_lead', 'vector_search_tool']
`persona_manager_agent` tools: ['buscar_conta', 'database_tool']
3. `ws-server` mescla MCP_TOOLS? True
4. `requirements.txt`: `mcp>=1.0.0`



Cada agente recebeu exatamente a tool MCP que lhe foi atribuída na Fase 2.

T4 — TESTE DEFINITIVO: invocar a tool MCP GERADA (chama o servidor MCP real)

Extraído o app e invocado o wrapper gerado diretamente:

```
import mcp_tools
tool = mcp_tools.MCP_TOOLS['enriquecer_lead']
tool._run(email='ana@medai.com.br')
```

Resultado:

```
{
  "email": "ana@medai.com.br",
  "empresa": "MedAI Diagnósticos",
  "cargo": "CTO",
  "porte": "120 func."
}
```

✅ A ferramenta MCP gerada pelo LangNet chamou o servidor MCP real e retornou o dado enriquecido. Este é o fechamento do ciclo: registrar (F1) → atribuir ao agente (F2) → o app gerado usa de verdade (F3).

T5 — ws-server sobe com MCP

O ws-server do app gerado sobe (porta 5002) com o `mcp_tools.py` presente, sem prompt interativo nem erro de import. ✅

4. Conclusão

Fase 3 **completa e validada ponta a ponta**. O pipeline do LangNet agora gera apps cujos agentes usam **ferramentas MCP reais** — plugando em sistemas externos do cliente sem programação manual, que é exatamente o negócio da Quântica (consultoria/dev que incorpora IA aos sistemas do cliente).

Ciclo MCP completo (F1 → F2 → F3): registrar servidor global → descobrir tools → habilitar no projeto → atribuir aos agentes (sugerido/manual) → Geração de Código emite `mcp_tools.py` + injeta nos agentes → **app gerado chama a ferramenta MCP real**.

Pendências (fora do escopo destas 3 fases)

- Transporte **stdio** (processo local) — hoje sse/http.
- Fase 4 (opcional): Sincronização de Estados MCP.